HACKTHEBOX

Meerkat Writeup



Updated: 2025-01-28

Prepared by: CyberJunkie & idna

Machine Author(s): sebh24

Difficulty: Easy

Description

> You are brought in as the new security provider for Forela, a fast-growing startup. The startup has been using a business management platform, but with insufficient documentation and potentially lax security practices. You are provided with PCAP and log data and are tasked with determining if a compromise has occurred. This scenario pushes the you to employ your analysis skills, effectively sifting through network data and logs to detect potential signs of intrusion, thereby offering a realistic taste of the pivotal role cybersecurity plays in the protection of burgeoning businesses.

Scenario

> As a fast-growing startup, Forela has been utilising a business management platform. Unfortunately, our documentation is scarce, and our administrators aren't the most security aware. As our new security provider we'd like you to have a look at some PCAP and log data we have exported to confirm if we have (or have not) been compromised.

Artifacts Provided

Challenge Files:

- meerkat.zip - a76867ade304c8081e2023cbf2977c65e8c146180b2e0ff760e4059d042c2a5a

```
┌─[user@HackThebox]─[~/htb/Sherlocks/Meerkat]
└──╼ $sha256sum meerkat.zip
a76867ade304c8081e2023cbf2977c65e8c146180b2e0ff760e4059d042c2a5a  meerkat.zip
```

Zip contents:

- meerkat.pcap - aa3838dbd634f9798d1e9505d243a4fee1d340d6e25e2f0c9648dd64e2178dbf
- meerkat-alerts.json - 012aa4e8aae5d500c001510d6e65567eb0cdbfffe2dab9a119b66f7770c222be

```
┌─[user@HackThebox]─[~/htb/Sherlocks/Meerkat]
└──╼ $sha256sum meerkat.pcap
aa3838dbd634f9798d1e9505d243a4fee1d340d6e25e2f0c9648dd64e2178dbf  meerkat.pcap
┌─[user@HackThebox]─[~/htb/Sherlocks/Meerkat]
└──╼ $sha256sum meerkat-alerts.json
012aa4e8aae5d500c001510d6e65567eb0cdbfffe2dab9a119b66f7770c222be  meerkat-alerts.json
```

Skills Learnt

- Network packet capture analysis with Wireshark and tshark
- Zeek alert analysis
- Parsing json data with jq
- Use of the "Wayback Machine" in Incident Response
- An understanding of different types of brute force attacks
- Alignment of the attack chain to MITRE ATT&CK

Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue`.

```
┌─[✗]─[user@HackThebox]─[~/htb/Sherlocks/Meerkat]
└──╼ $unzip -P hacktheblue meerkat.zip
Archive:  meerkat.zip
  inflating: meerkat.pcap
  inflating: meerkat-alerts.json
```

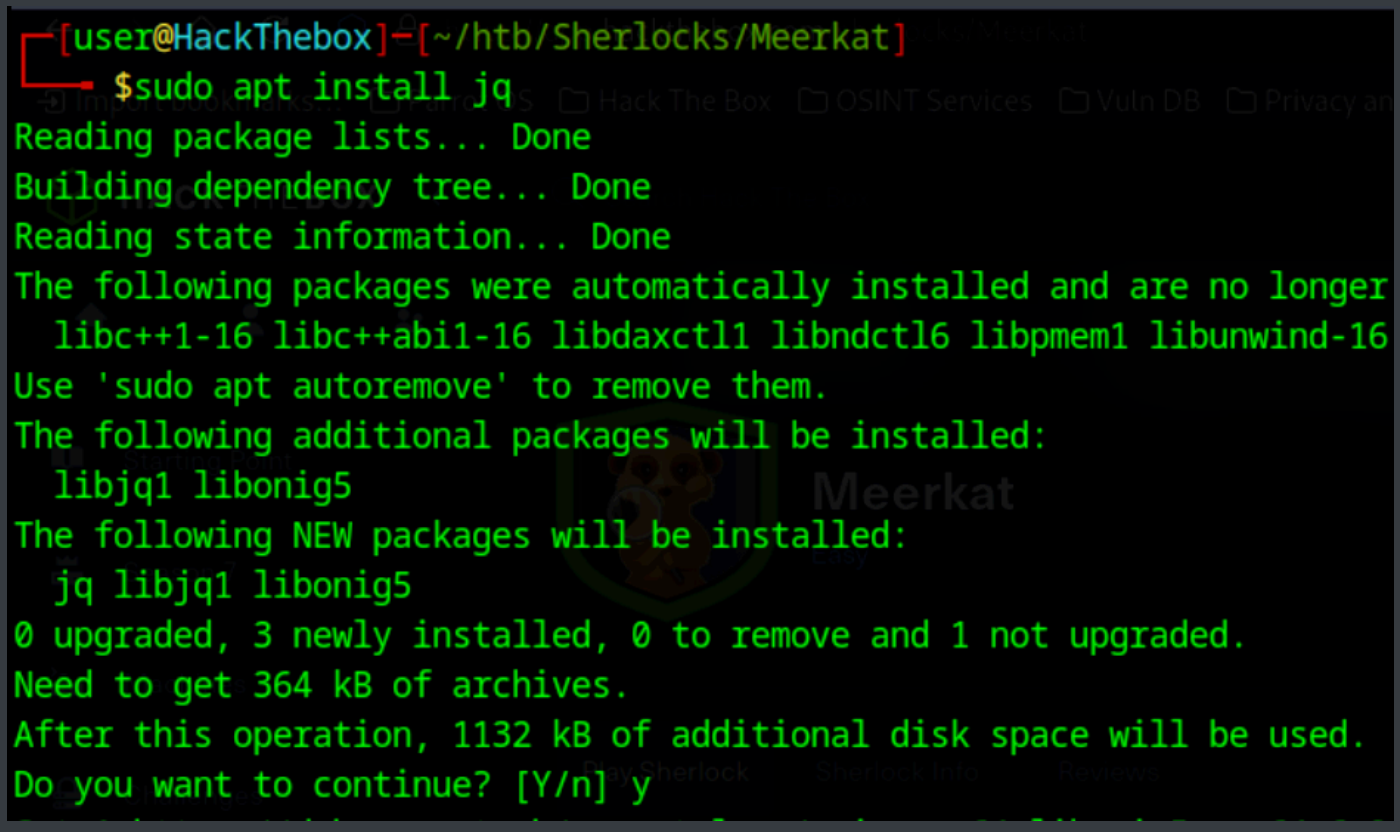
Zeek alerts

Zeek is a passive, open-source network traffic analyzer tool. It analyzes network traffic packets and creates “Zeek logs” which can be used to detect malicious activity within a network. Different types of network metadata, like Zeek logs, IPFIX and NetFlow, allow better visibility into specific network conversations (or flows).

The json file here is a series of Zeek alerts that have been output in json format. The Zeek documentation describes ways to parse this using the 'jq' tool. This tool allows many ways to manipulate, search and filter the content to find what we're looking for.

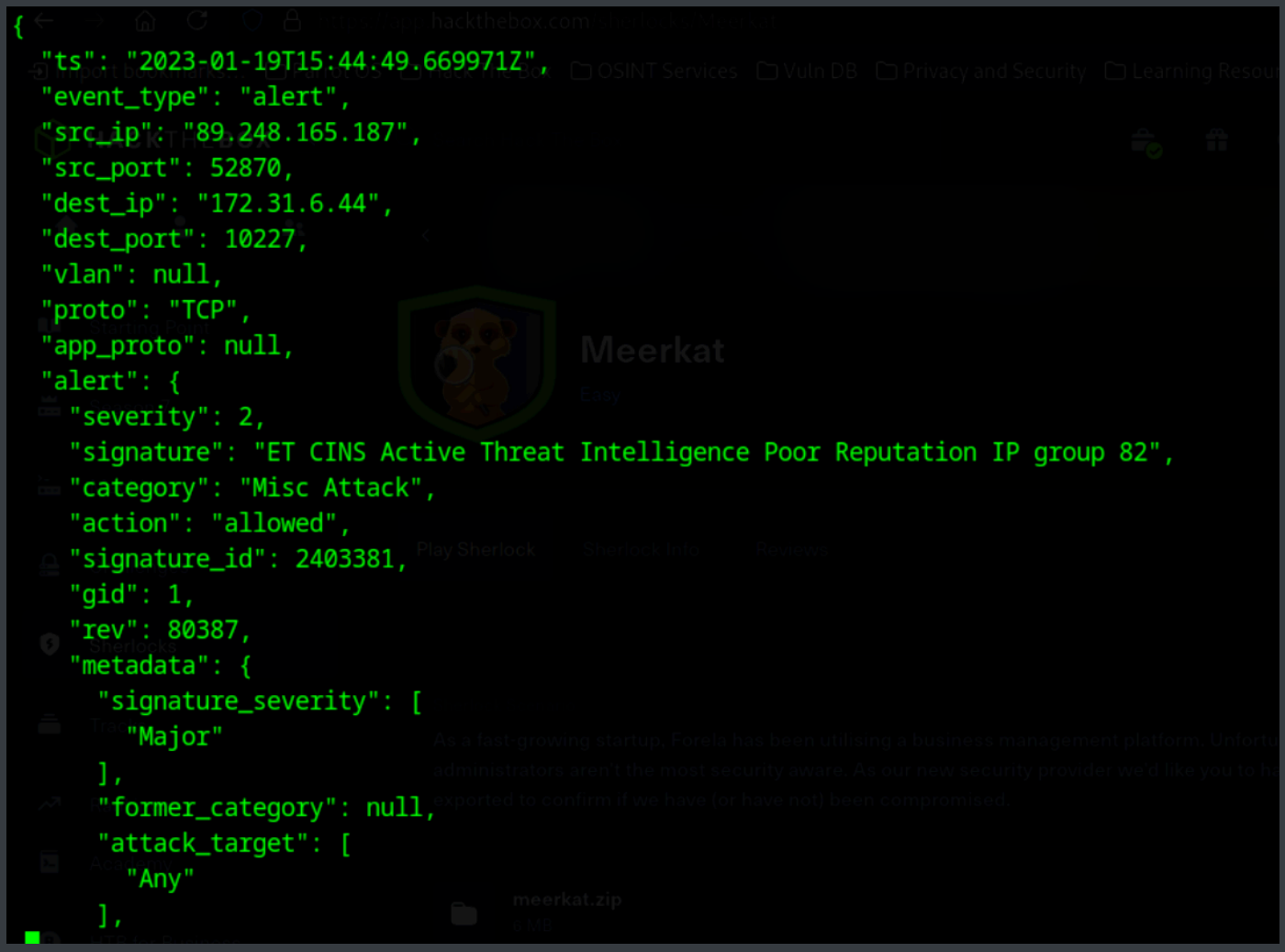
To begin we will install jq using:

```
sudo apt install jq
```



We can now start to investigate the alerts using the following command to firstly remove the outer json list and secondly pipes the command into less allows us to navigate through the output.

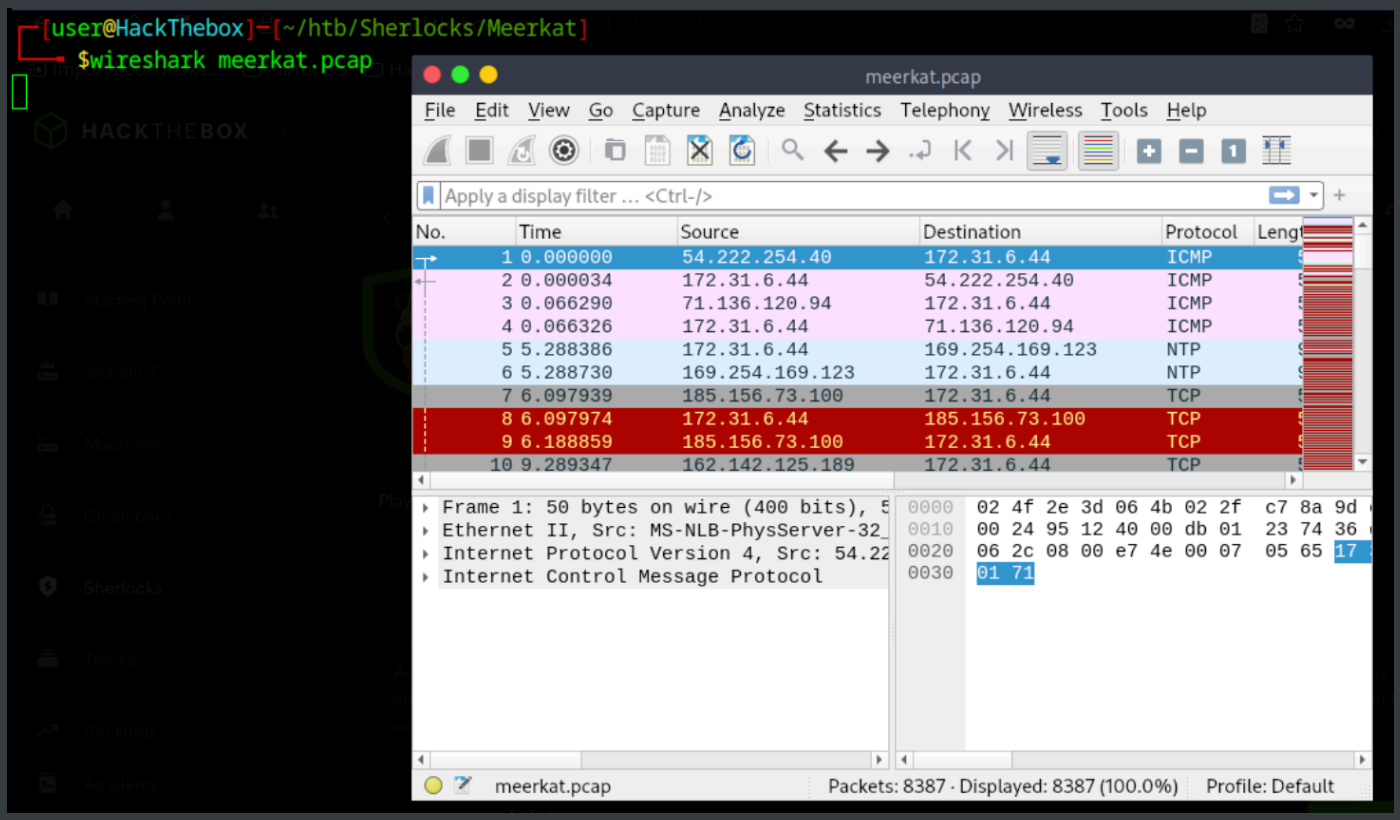
```
jq '.[]' meerkat-alerts.json | less
```



Packet Capture

The pcap file is a network packet capture. There are many ways to capture such a file including the incredibly popular Wireshark.

To open the pcap in Wireshark on ParrotOS or other similar operating systems use `wireshark meerkat.pcap`

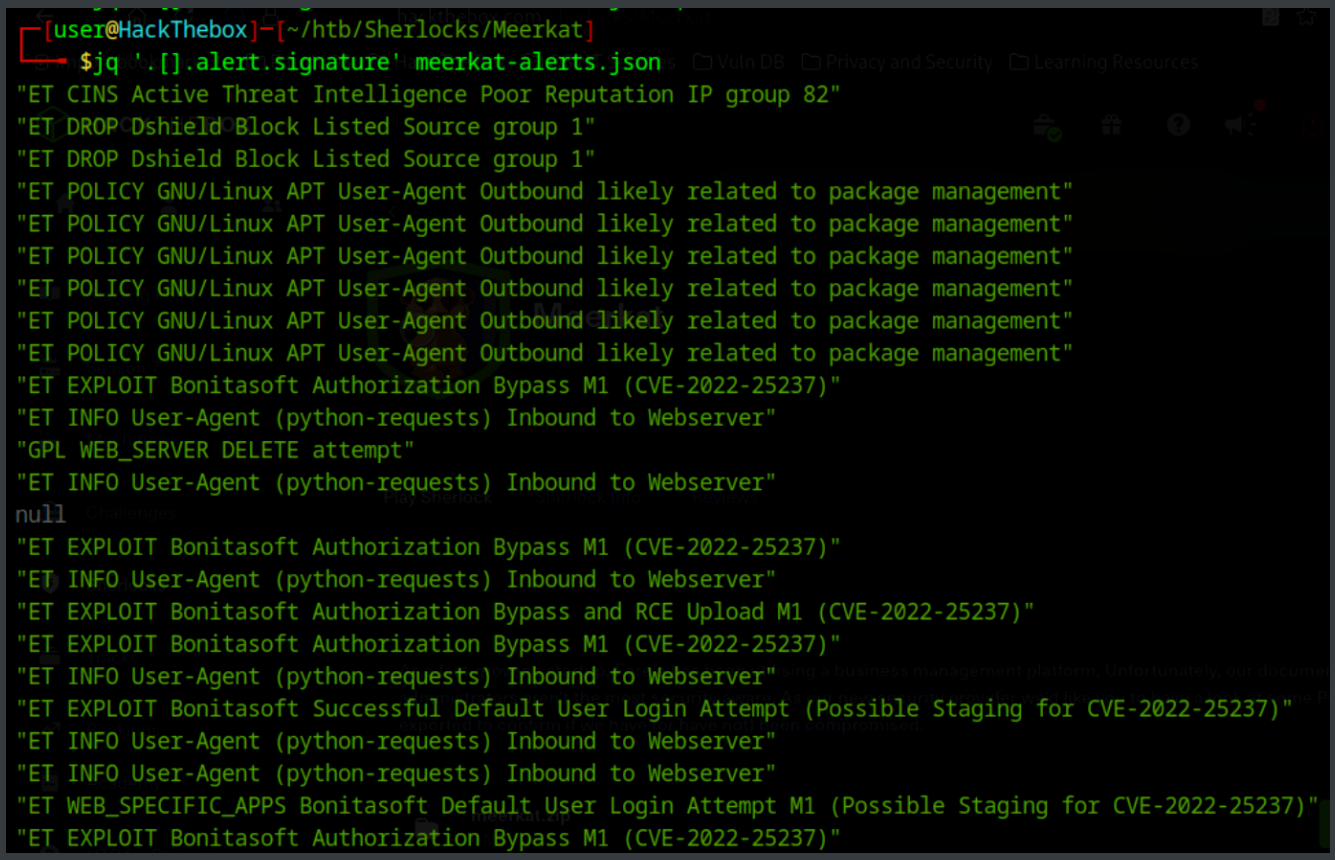


Questions

1. **We believe our Business Management Platform server has been compromised. Please can you confirm the name of the application running?**

   We can use the jq command we saw earlier to parse the Zeek alerts. To make things simpler we can add a filter to only display the alert names. This shows us the inital attack flow and what seems to be the successful exploitation of Bonitasoft, which according to their website is a Business process management application.

   ```
   jq '.[].alert.signature' meerkat-alerts.json
   ```



Answer: `BonitaSoft`

2. **We believe the attacker may have used a sub-technique of the brute forcing attack technique - what is the name of the attack carried out?**

   The vulnerability seen in question 1 is explained here: https://rhinosecuritylabs.com/application-security/cve-2022-25237-bonitasoft-authorization-bypass/

   The exploit relies on a finding a legitimate username and password combo to allow privilege escalation and ultimately code execution as long as that user has a valid session token at the time of the exploit. This does indicate that an attacker would potentially look to use a Brute Forcing technique on the application to find some valid credentials, but doesn't tell us enough about what type of brute force attack was taking place.

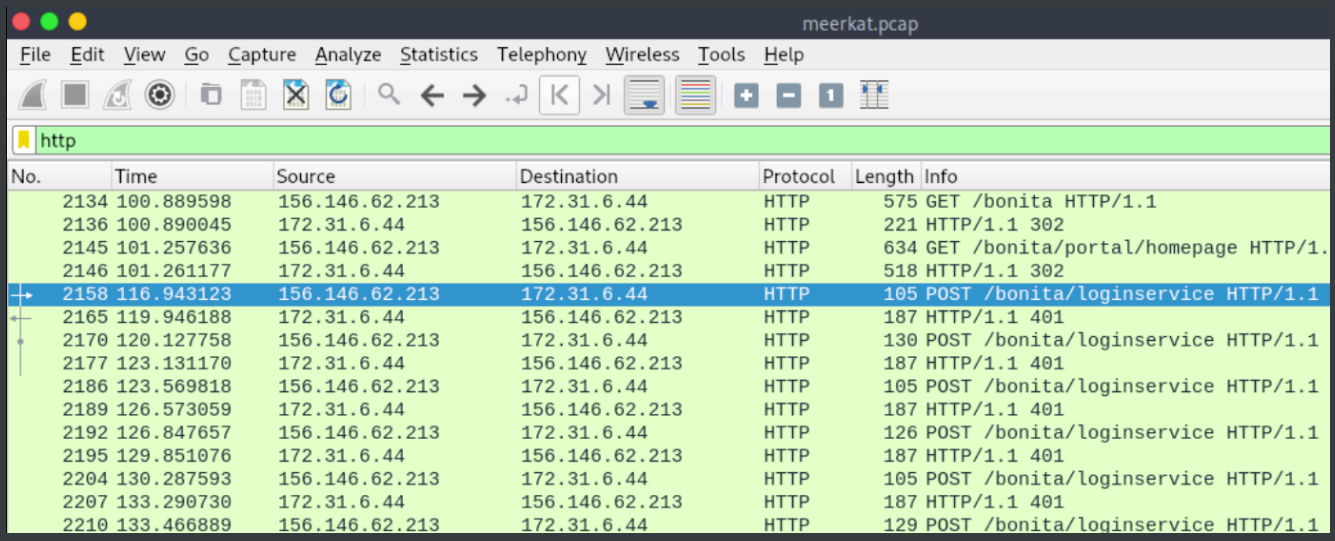
   Looking at the MITRE ATT&CK page on the Brute Force technique we see it has the following sub-techniques:

   - Password Guessing: Adversaries with no prior knowledge of legitimate credentials within the system or environment may guess passwords to attempt access to accounts.

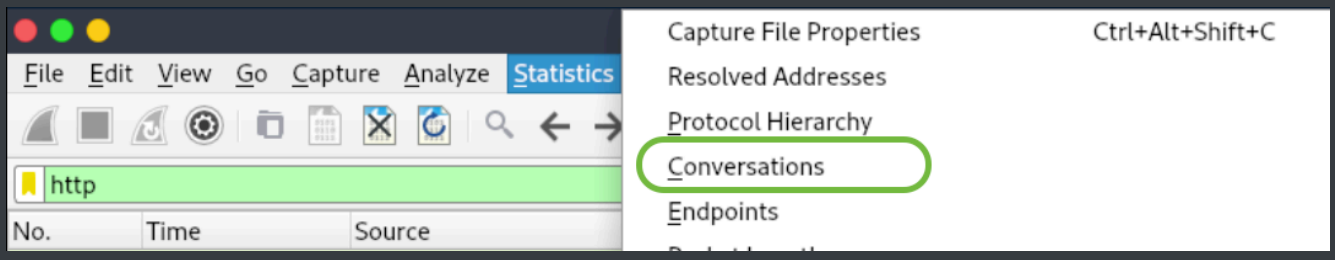
- Password Cracking: Adversaries may use password cracking to attempt to recover usable credentials, such as plaintext passwords, when credential material such as password hashes are obtained.
- Password Spraying: Adversaries may use a single or small list of commonly used passwords against many different accounts to attempt to acquire valid account credentials.
- Credential Stuffing: Adversaries may use credentials obtained from breach dumps of unrelated accounts to gain access to target accounts through credential overlap.

In order to answer the question we will need to see what username and password combinations are being used.

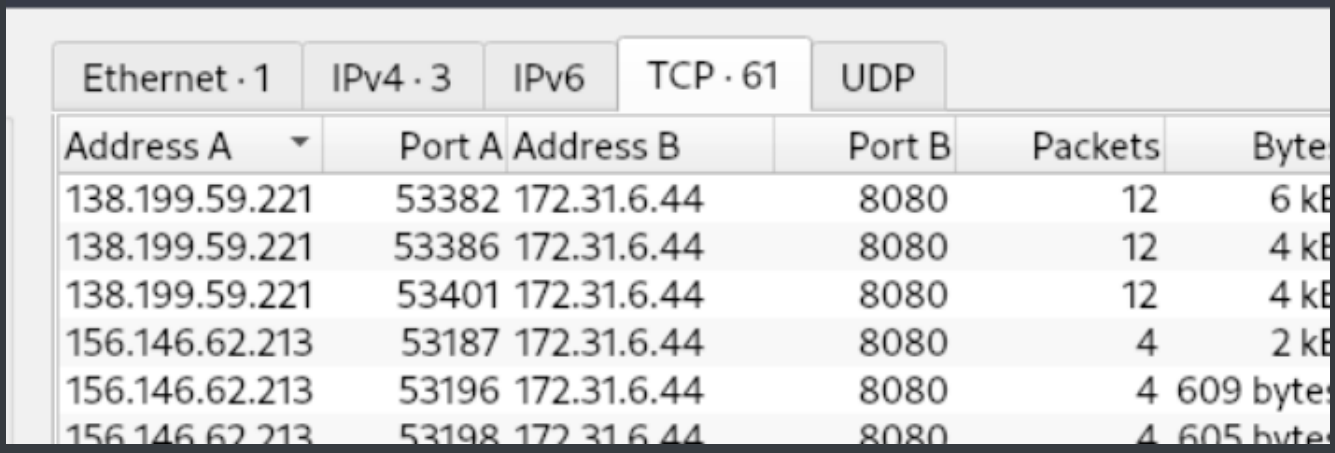
We can see the exploit attempts by opening the pcap in Wireshark and filtering for http traffic.



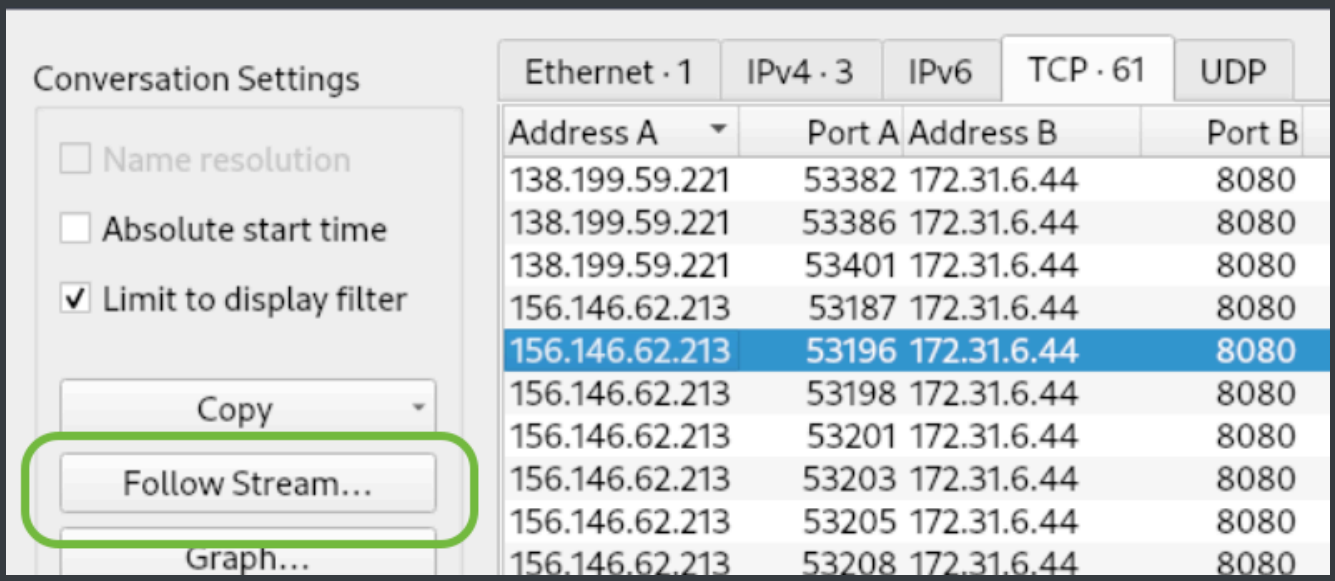
From here we go to the Statistics>Conversations menu to see all of the network conversations grouped together:

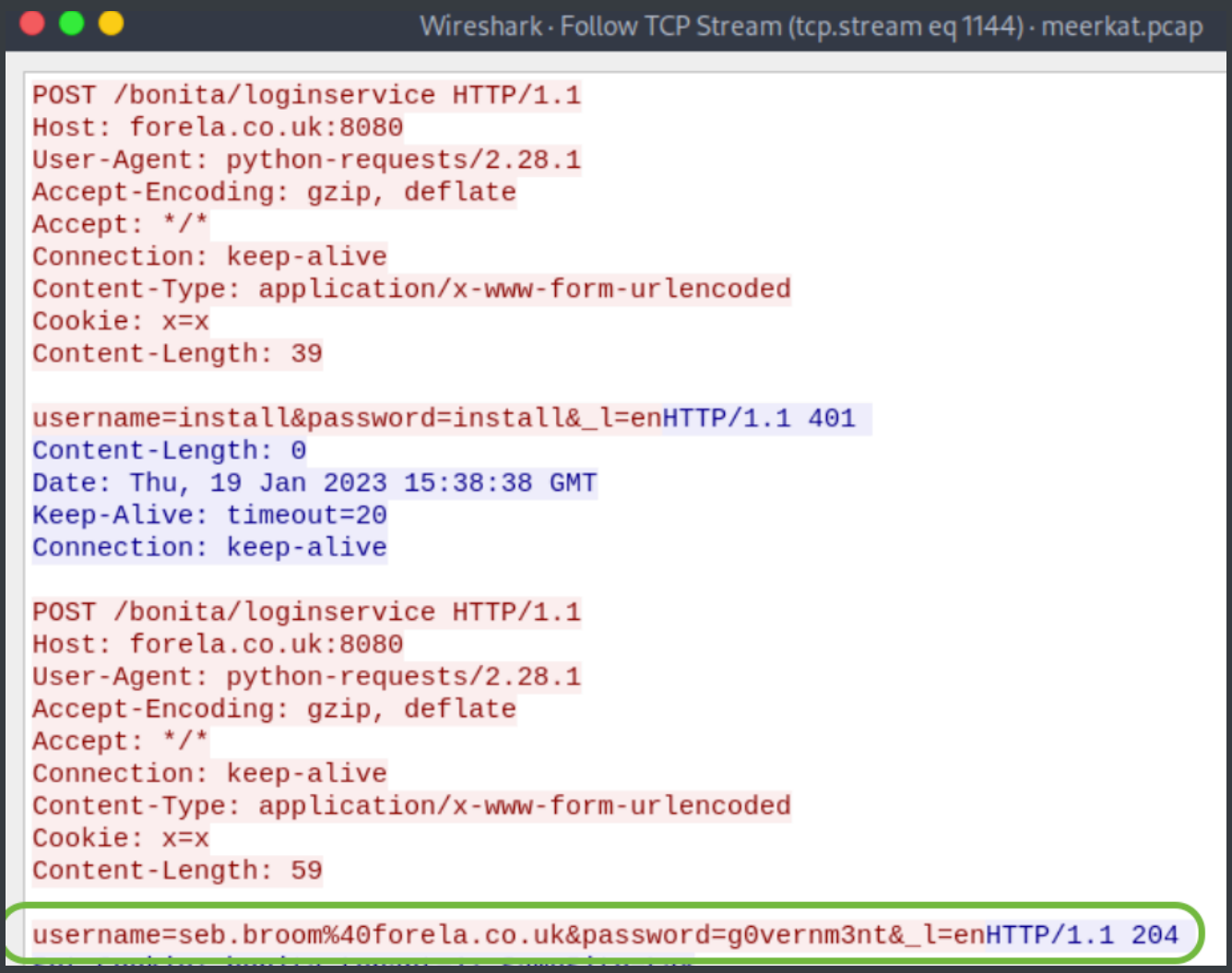


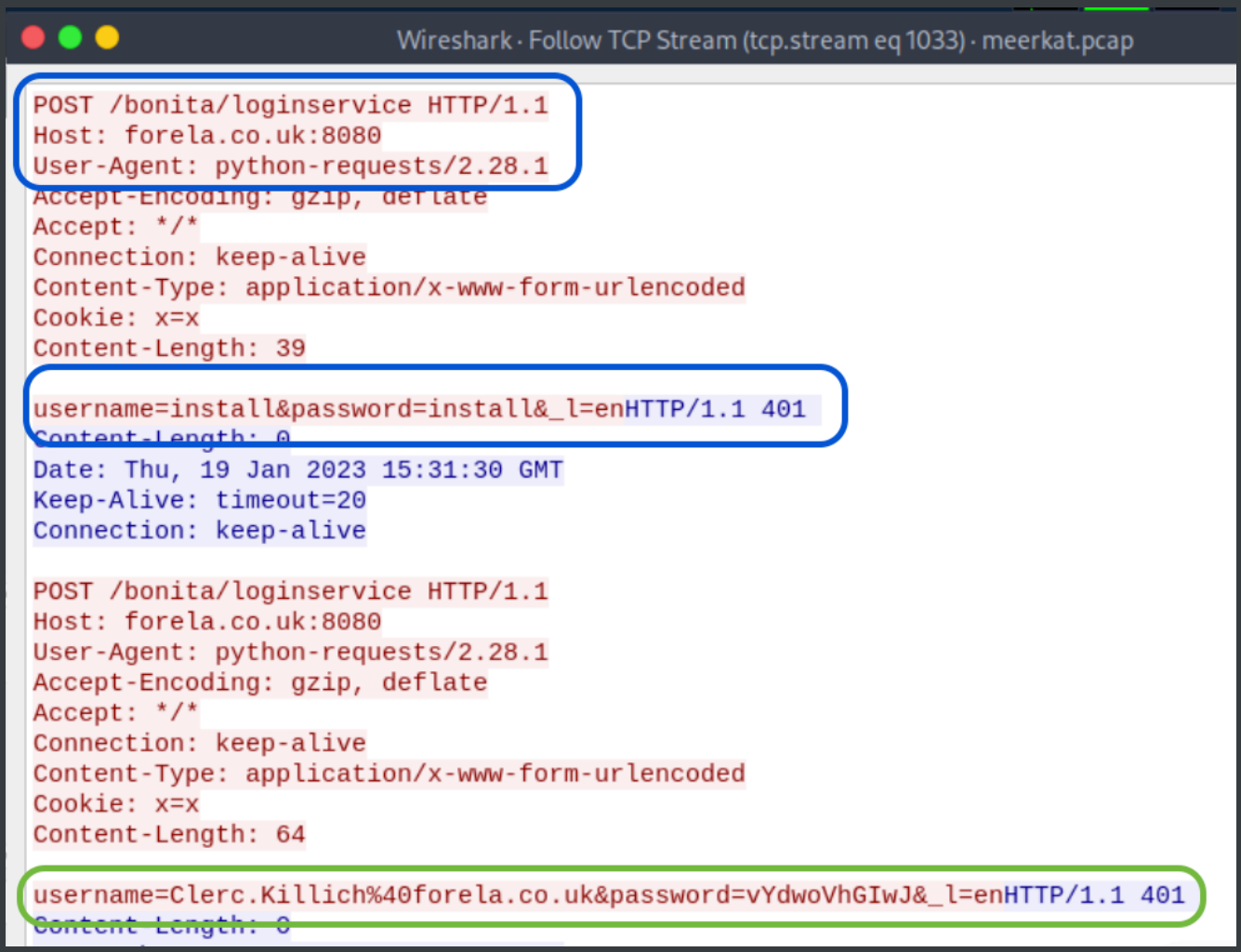
As we want to see http conversations we can look at the TCP tab



We can see that most of the conversations come from 138.199.59.221 towards 172.31.6.44 on port 8080. As this traffic is not encrypted using https we will be able to see the packet details. By selecting different samples of these conversations and choosing 'Follow Stream' we can see the credentials being attempted by the attacker on each attempt.

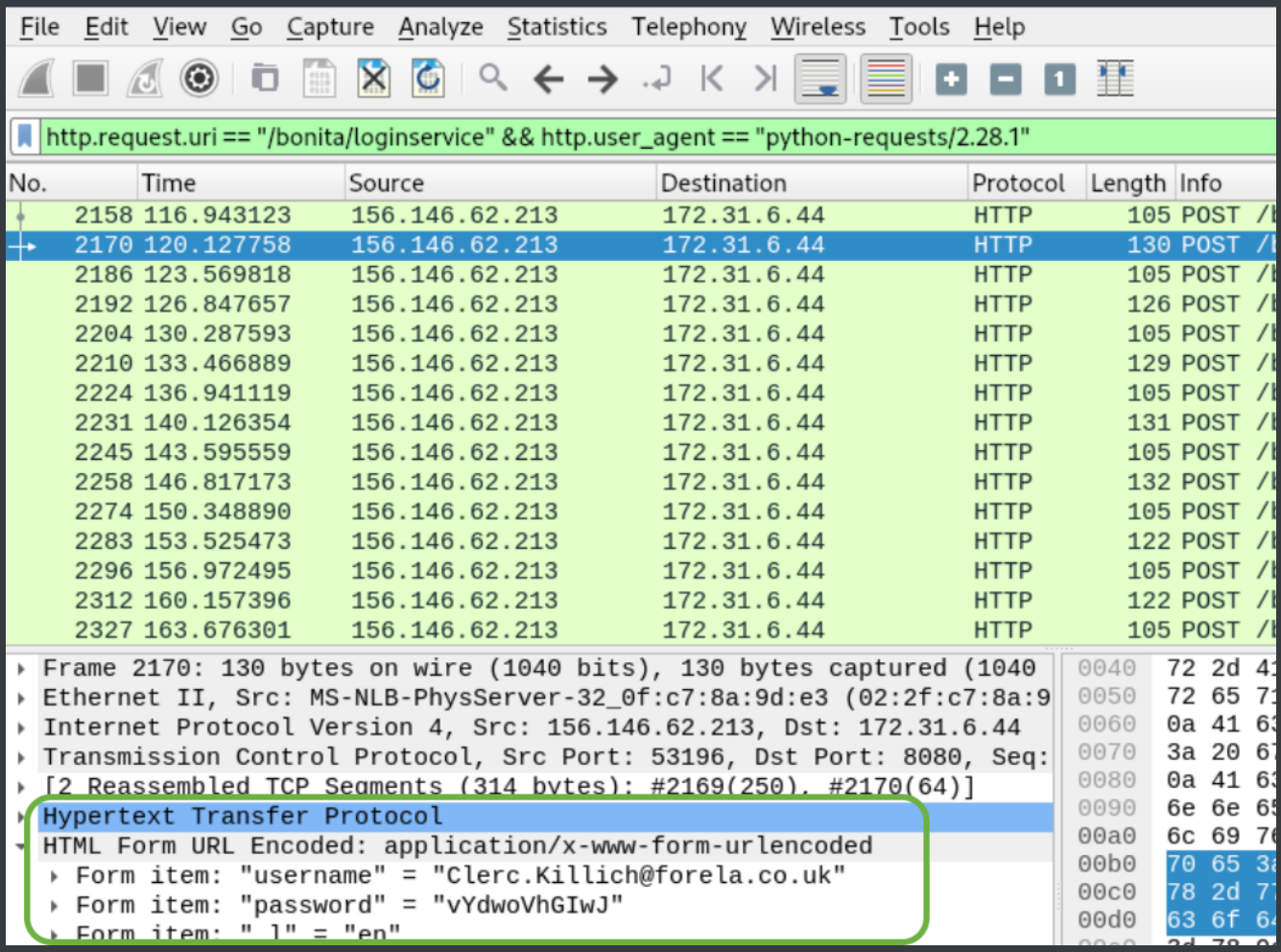



Wireshark · Follow TCP Stream (tcp.stream eq 1144) · meerkat.pcap
POST /bonita/loginservice HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 39
username=install&password=install&_l=enHTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:38:38 GMT
Keep-Alive: timeout=20
Connection: keep-alive
POST /bonita/loginservice HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 59
username=seb.broom%40forela.co.uk&password=g0vernm3nt&_l=enHTTP/1.1 204



We can also see the username and password used in the attack back in the main Wireshark window in the Packet Details pane. Using the detail in some of the blue boxes pictured in the screenshot above, we can apply a wireshark display filter to see just the exploit attempts:

```
http.request.uri == "/bonita/loginservice" && http.user_agent ==
"python-requests/2.28.1"
```



We could choose to look at each individual conversation or packet to see the usernames and passwords being used, however we will use another tool from the developers of Wireshark, called tshark. This is a command line version of Wireshark which will allow us to see all of this in one go.

We will use the command below to see each username and password combination used:

```
tshark -Y '(http.request.uri == "/bonita/loginservice") &&
(http.user_agent == "python-requests/2.28.1")' -r meerkat.pcap -T
json | jq '.[]._source.layers.http."http.file_data"' | sort | uniq
```

We can break down this command as follows:

`-Y` to add the same display filter we used in Wireshark above

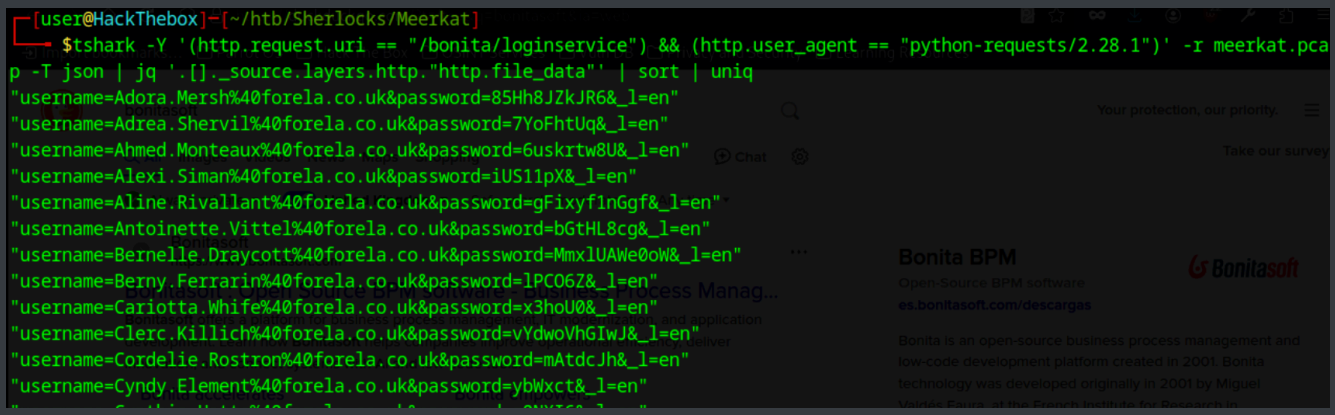
`-r` reads in the pcap file

`-T json` converts the output to json

We then use the `|` to pipe the output into jq, the same tool we used earlier to parse the Zeek json file, this time with a filter that keeps telling jq to drill into the key that contains the data we need to extract:

`'.[]'` firstly tells jq to drill into the main json list output, then the `_source` key, then `layers`, then `http` and finally the value we want is inside the `file_data` key

We then pipe this again into the `sort` command and finally `uniq`. These two commands together are a common way to extract only the unique entries.



Finally we can see the username and password combinations the attacker was using. Thinking back to the definitions of the brute force attack types earlier we can make the following determination:

- The attack **does not** appear to be using completely guessed username and password combinations, therefore we can rule out Password Guessing
- The attack **does not** resemble any kind of attempt to crack or expose password hashes, therefore we can rule out Password Cracking
- The attack **is not** attempting the same password across multiple usernames, therefore we can rule out Password Spraying
- The attack **does** look to be using valid looking usernames, and uncommon passwords. This could suggest that the attacker is using a password list based on credentials found on the dark web and is seeing if anyone has reused the same password combination in this application. This does mean the attack looks like a Credential Stuffing attack.

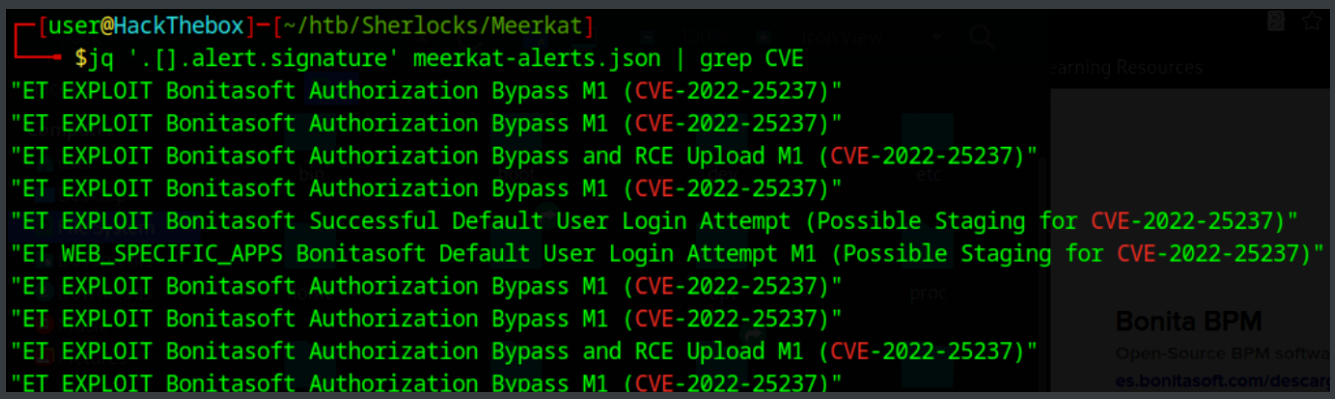
Answer: `Credential Stuffing`

3. **Does the vulnerability exploited have a CVE assigned - if so, which one?**

When we investigated the Zeek alerts earlier we saw the Bonita vulnerability did have a CVE assigned.

We can see this again filtering the output to just alerts continaining 'CVE' using:

```
jq '.[].alert.signature' meerkat-alerts.json | grep CVE
```



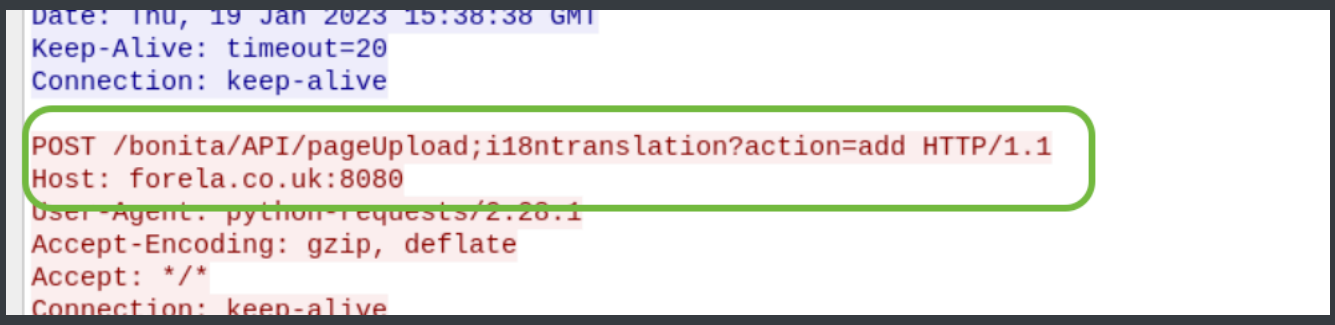
Answer: `CVE-2022-25237`

4. **Which string was appended to the API URL path to bypass the authorization filter by the attacker's exploit?**

   The vulnerability is explained here: https://rhinosecuritylabs.com/application-security/cve-2022-25237-bonitasoft-authorization-bypass/
   This shows the string to be 'i18ntranslation'.

   We can also see this in the packet capture by following the http stream of one of the exploit attempts in the same way we did earlier.

   Here we can see the API endpoint to be /bonita/API/pageUpload and this is appended with `;i18ntranslation`:

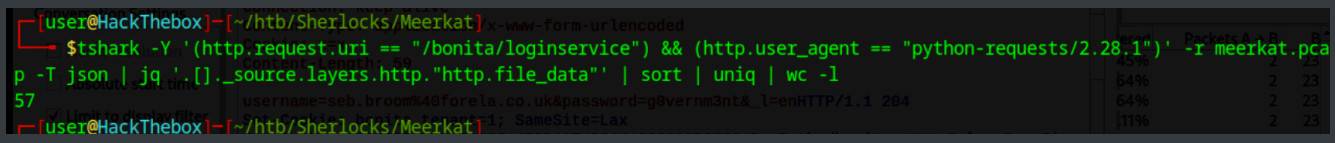


Answer: `i18ntranslation`

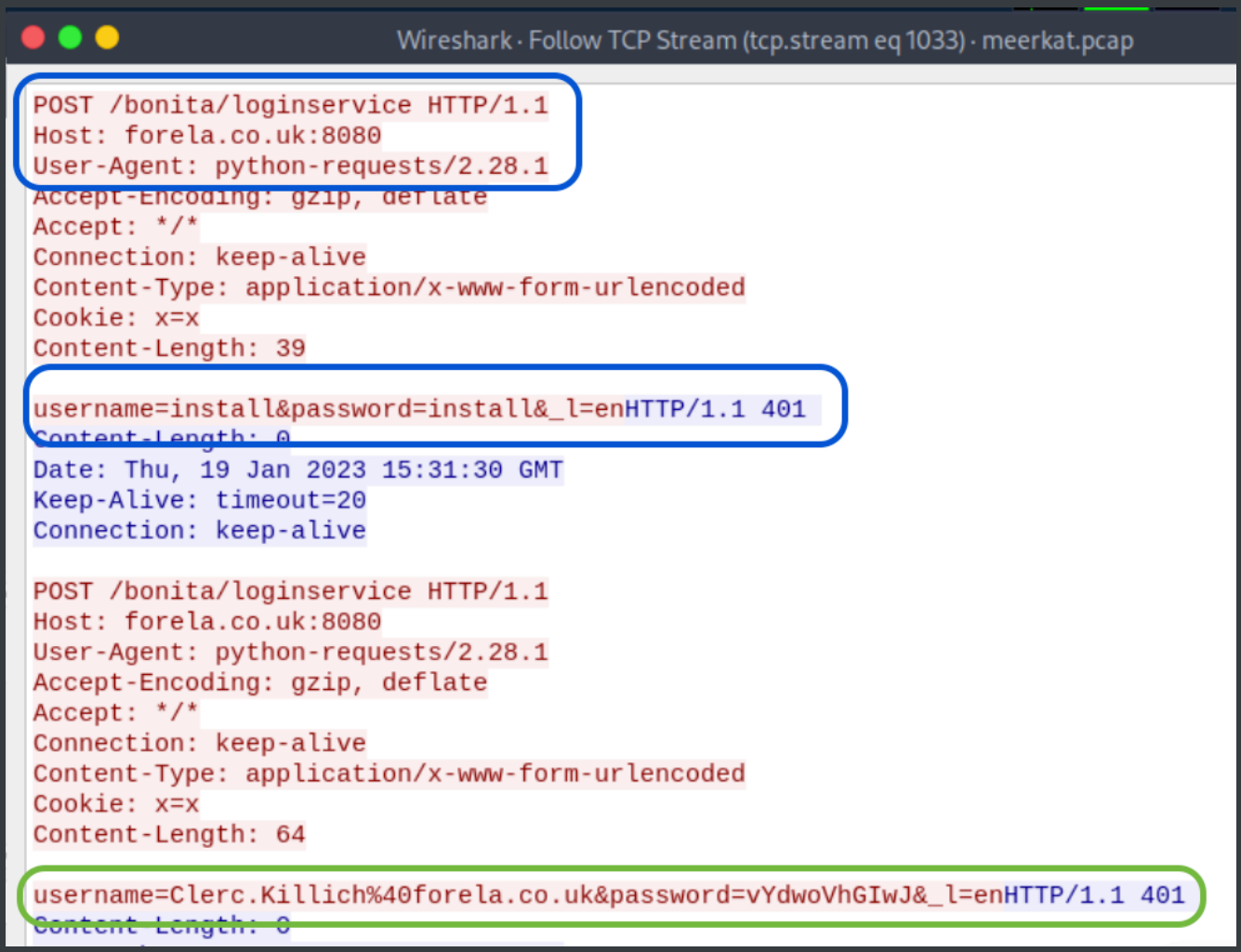
5. **How many combinations of usernames and passwords were used in the credential stuffing attack?**

   Looking back at the tshark command in question 2 we saw a list of all of the unique usernames and passwords used. We can use that same command again, and add an additional pipe to count the lines output using `wc -l`:

```
tshark -Y '(http.request.uri == "/bonita/loginservice") &&
(http.user_agent == "python-requests/2.28.1")' -r meerkat.pcap -T
json | jq '.[]._source.layers.http."http.file_data"' | sort | uniq
| wc -l
```



   That gives us 57. **However** if looking at the exploit code here we can see that the exploit always tries the default credentials of `install:install` each time it is run. We can see evidence of this in the packet capture too. In the screenshot below, one of the blue boxes shows the default credentials being attempted and this is visible in every attempt:



Therefore, as this wasn't actually being tried by the attacker as part of their Credential Stuffing attack, we can subtract 1 from our total of the unique username and password counts that our tshark command gave us, which gives us the answer of 56.

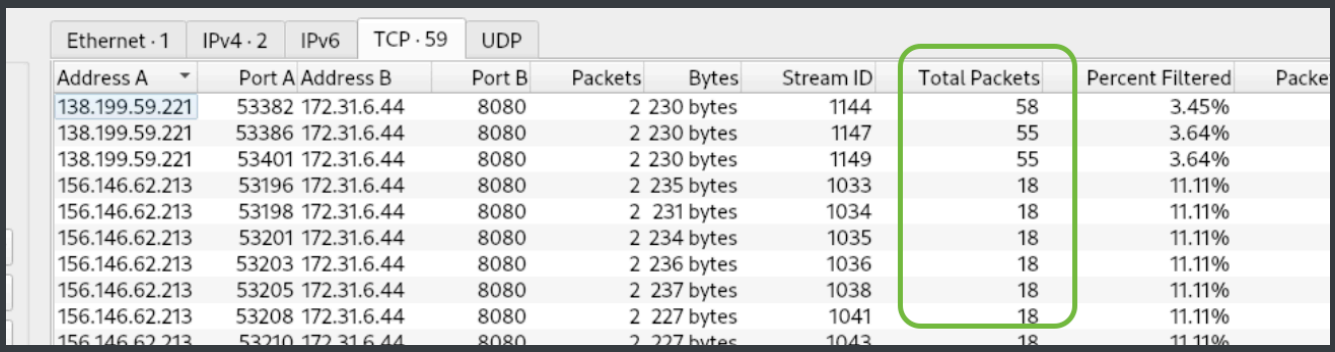
Answer: 56

6. **Which username and password combination was successful?**

   Back in Wireshark, filter on the endpoint we know was targetted:

   ```
   http.request.uri == "/bonita/loginservice"
   ```

   Once again look at Statistics > Conversations menu. The conversations are sorted by default by the total number of packets in each.



Ethernet · 1 | IPv4 · 2 | IPv6 | TCP · 59 | UDP

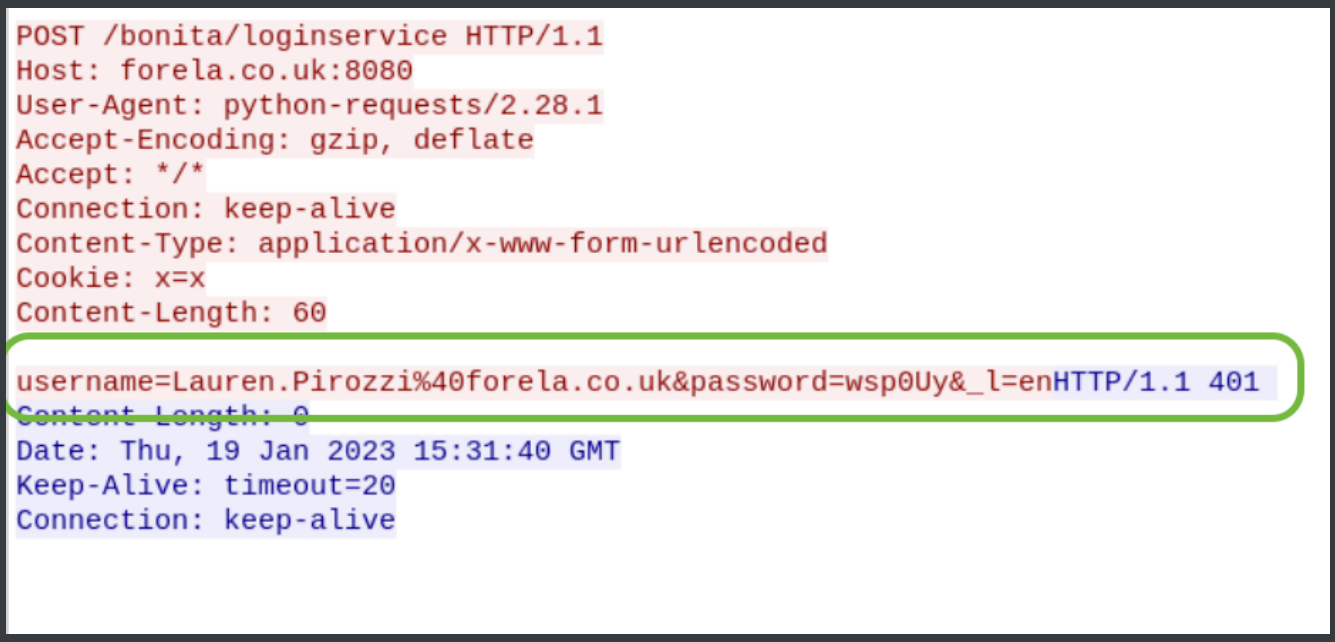
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packe
138.199.59.221	53382	172.31.6.44	8080	2	230 bytes	1144	58	3.45%	
138.199.59.221	53386	172.31.6.44	8080	2	230 bytes	1147	55	3.64%	
138.199.59.221	53401	172.31.6.44	8080	2	230 bytes	1149	55	3.64%	
156.146.62.213	53196	172.31.6.44	8080	2	235 bytes	1033	18	11.11%	
156.146.62.213	53198	172.31.6.44	8080	2	231 bytes	1034	18	11.11%	
156.146.62.213	53201	172.31.6.44	8080	2	234 bytes	1035	18	11.11%	
156.146.62.213	53203	172.31.6.44	8080	2	236 bytes	1036	18	11.11%	
156.146.62.213	53205	172.31.6.44	8080	2	237 bytes	1038	18	11.11%	
156.146.62.213	53208	172.31.6.44	8080	2	227 bytes	1041	18	11.11%	
156.146.62.213	53210	172.31.6.44	8080	2	227 bytes	1043	18	11.11%	

If we follow the stream of one of the conversations with over 50 packets, and compare it with one with just 18, we can see that the conversations with more packets do show the attacker's performing the exploitation of the vulnerability, and those with less show no further activity after the attempt. Additionally those with less packets show a HTTP code of 401 which means "unauthorized".

```
username=seb.broom%40forela.co.uk&password=g0vernm3nt&_l=enHTTP/1.1 204
Set-Cookie: bonita.tenant=1; SameSite=Lax
Set-Cookie: JSESSIONID=0AD5E14F8D1AE496444835639D0E60A9; Path=/bonita; HttpOnly; SameSite
=Lax
Set-Cookie: X-Bonita-API-Token=8b71f28f-31aa-47cb-92b6-50521cd5bf92; Path=/bonita; SameSi
te=Lax
Set-Cookie: BOS_Locale=en; Path=/; SameSite=Lax
Date: Thu, 19 Jan 2023 15:38:38 GMT
Keep-Alive: timeout=20
Connection: keep-alive

POST /bonita/API/pageUpload;i18ntranslation?action=add HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Cookie: JSESSIONID=0AD5E14F8D1AE496444835639D0E60A9; X-Bonita-API-Token=8b71f28f-31aa-47c
b-92b6-50521cd5bf92; bonita.tenant=1; BOS_Locale=en
Content-Length: 15163
Content-Type: multipart/form-data; boundary=f79968fcad082cab61a2fc649dab366c

--f79968fcad082cab61a2fc649dab366c
Content-Disposition: form-data; name="file"; filename="rce_api_extension.zip"
Content-Type: application/octet-stream
```



Therefore we can assume the username and password used in the larger conversations was correct.

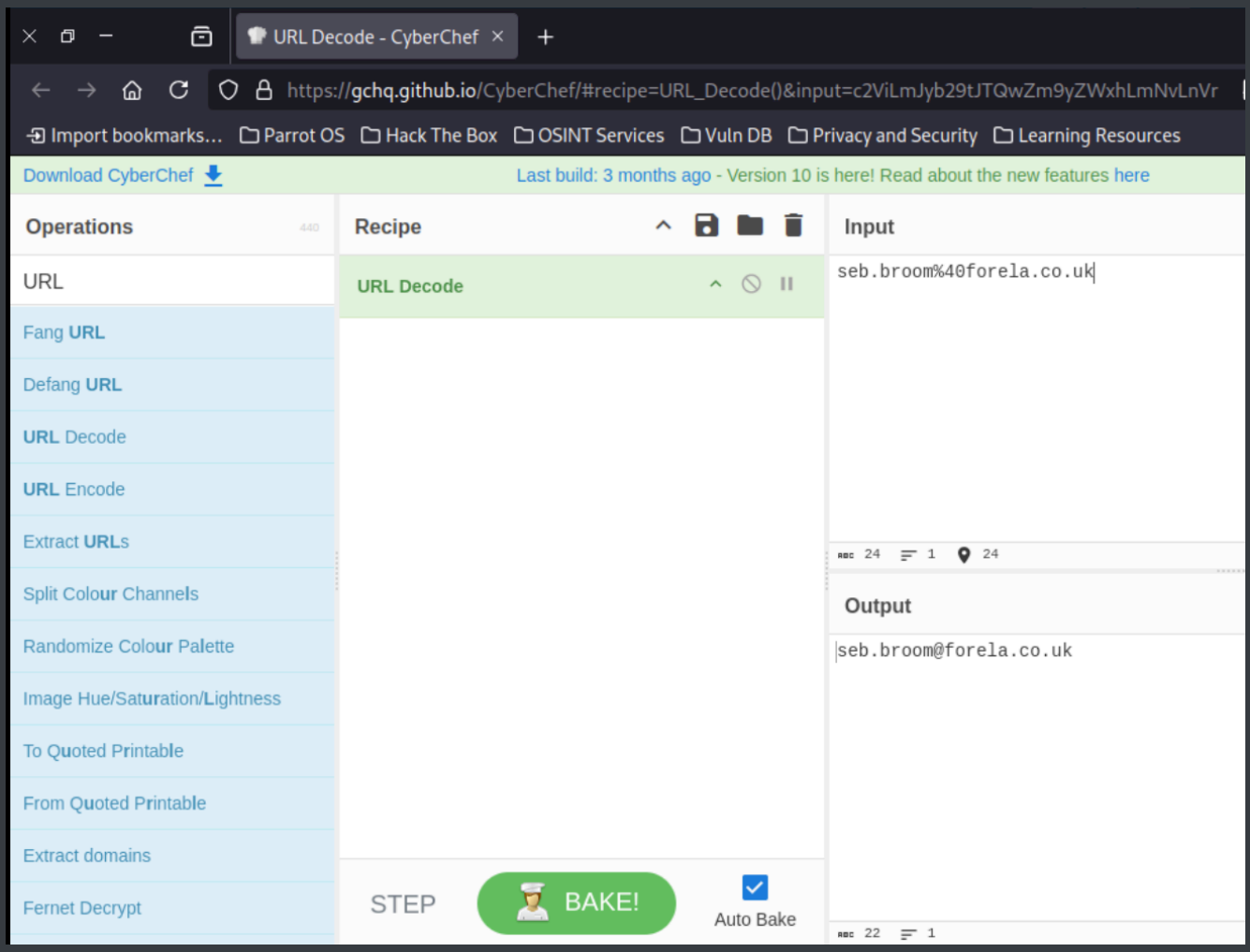
Username: seb.broom%40forela.co.uk

Password: g0vernm3nt

It is important to realise that as these were part of a HTTP request they are URL encoded. We will need to decode the username to reveal the special character. For this we can use Cyber Chef.

In the Operations window on the left search for URL Decode and drag it into the Recipe window in the middle of the page.

In the Input window on the top right enter the username found in Wireshark and click BAKE! to see the real username in the Output windows on the bottom right.



Answer: `seb.broom@forela.co.uk:g0vernm3nt`

7. **If any, which text sharing site did the attacker utilise?**

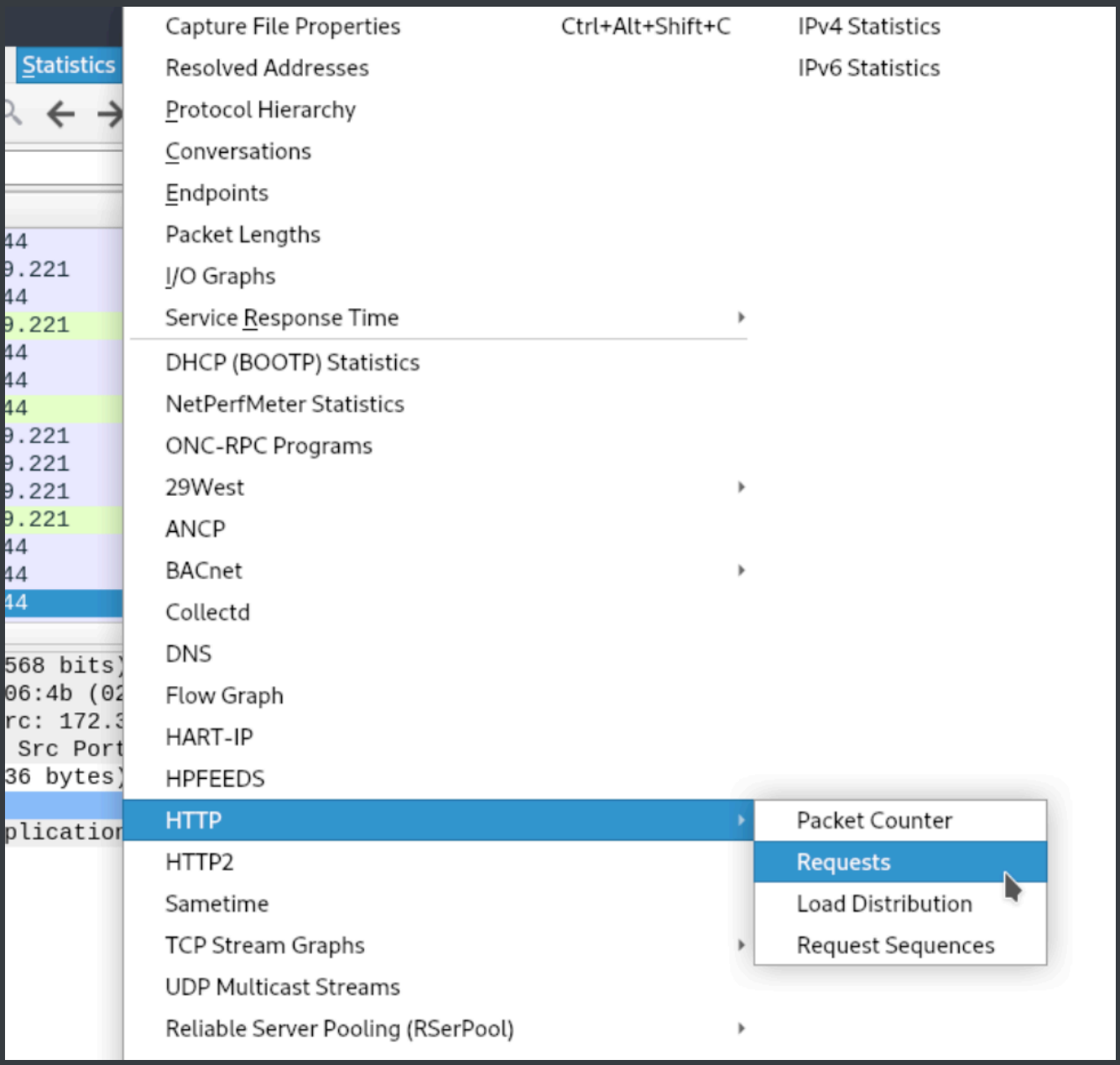
   Once again, by following the streams of the http conversations and focussing on those with the most packets as we did earlier, we can see the attackers commands being added to a GET request.

```
17a
{"processDefinitionId":"","updatedBy":"-1","urlToken":"custompage_resou
isplayName":"RCE","lastUpdateDate":"2023-01-19 15:38:38.792","descripti
anage resourceName","creationDate":"2023-01-19 15:38:38.792","contentNa
9936328099.zip","isHidden":"false","createdBy":"","isProvided":"false",
tType":"apiExtension"}
0

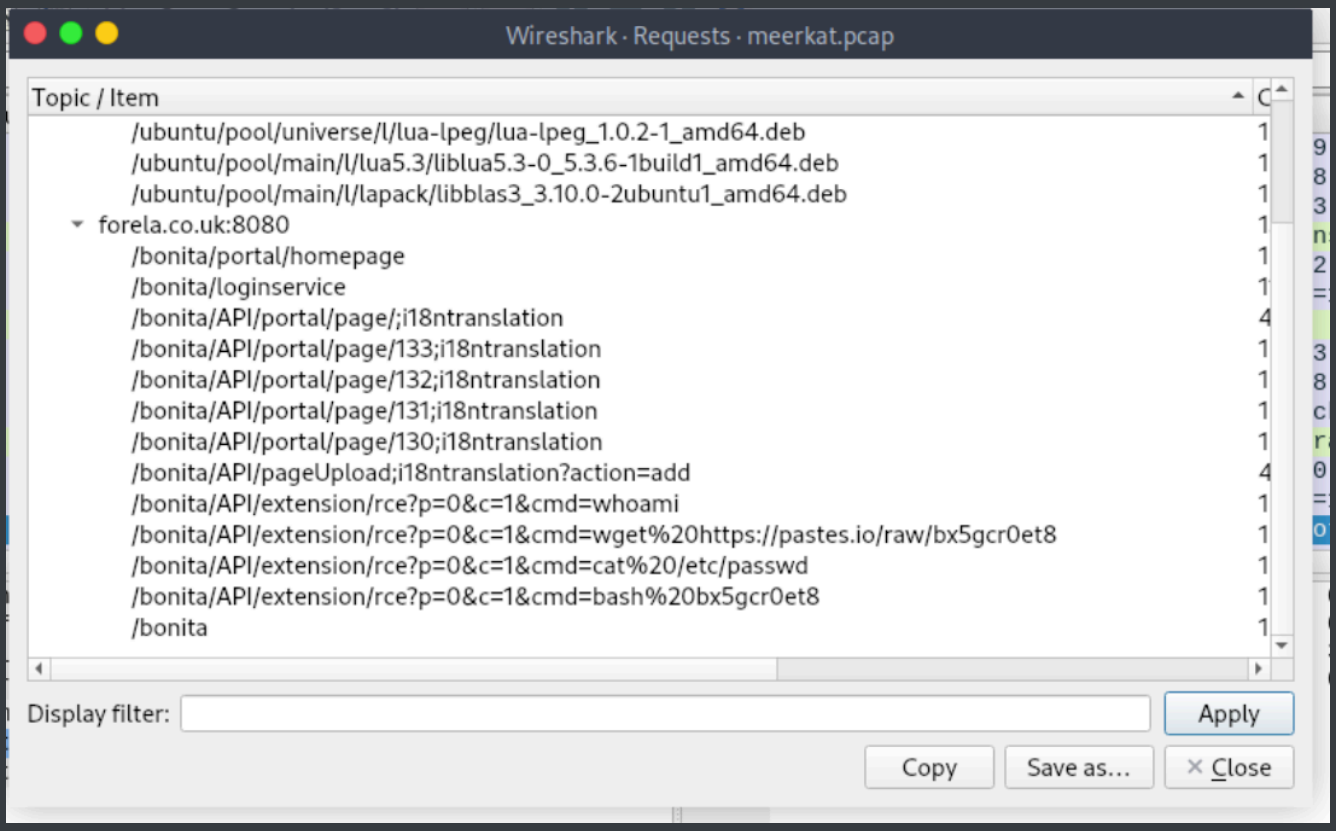
GET /bonita/API/extension/rce?p=0&c=1&cmd=cat%20/etc/passwd HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

As the commands are part of a GET request we can see the details in another way in Wireshark, which makes the list of commands run easier to see.

In Wireshark go to Statistics > HTTP > Requests



Here we can see the list of commands that the attacker ran using the exploit:

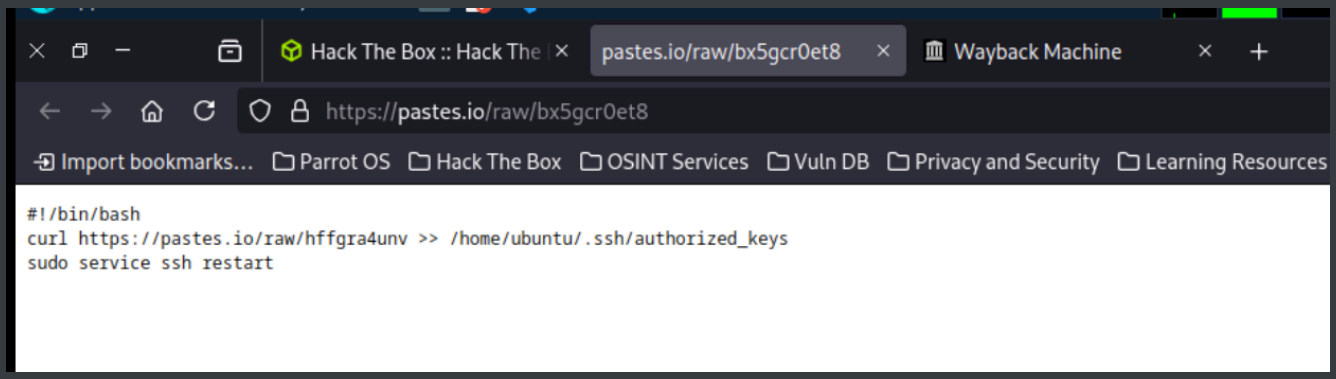


This shows us that after running `whoami` the attacker ran a command to download a file from 'pastes.io'.

Answer: `pastes.io`

8. **Please provide the filename of the public key used by the attacker to gain persistence on our host.**

   From the question above we know that the attacker downloaded the next stage of the payload from pastes.io. By browsing to this page we can see the contents of the file.

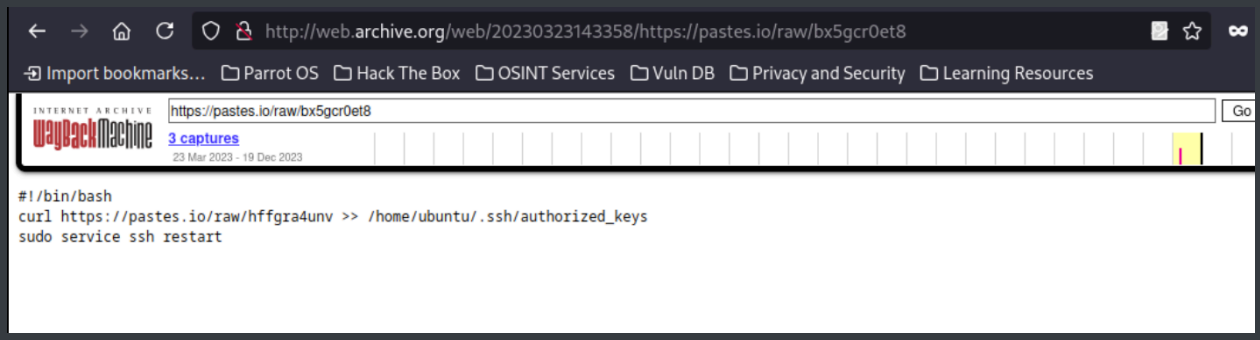


> **Note**

> In circumstances where we want to investigate an artifact that was hosted on the public internet, but is no longer available, we can use The Wayback Machine. This is useful to know in a real investigation where an attacker has removed their payloads.
>
> In this example, if the file is not present we can search this site for the URL and it will show us if it has archived a copy.



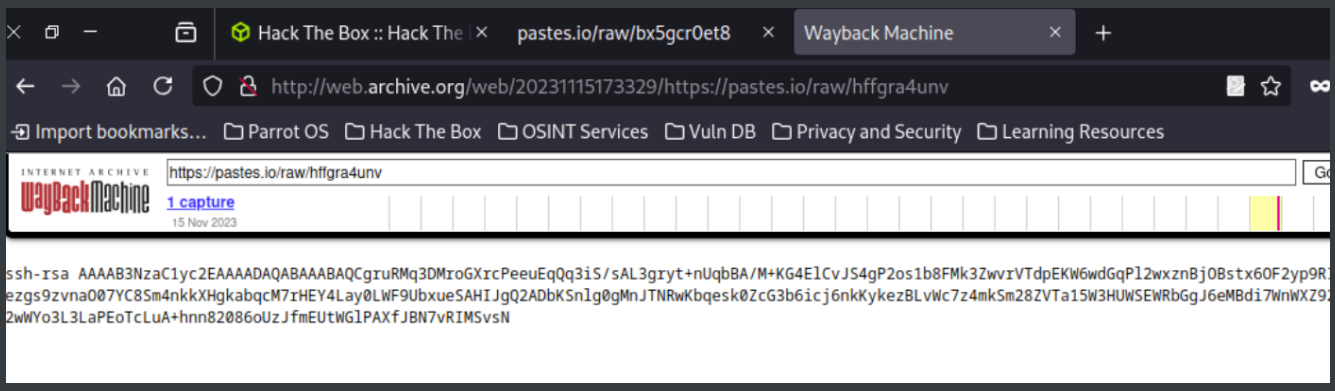
> Selecting the copy saved on March 23rd in the above picture we can see the contents of the file.



> Looking at this payload we can see the attacker is copying the contents of the file named 'hffgra4unv' into the 'authorised_keys' file.
>
> **Answer:** `hffgra4unv`

9. **Can you confirm the file modified by the attacker to gain persistence?**

   Using the Wayback machine again we can see the contents of this file to be the attackers SSH public key. Adding this to the 'authorized_keys' file will allow them to login to the server via SSH, therefore gaining persistence.

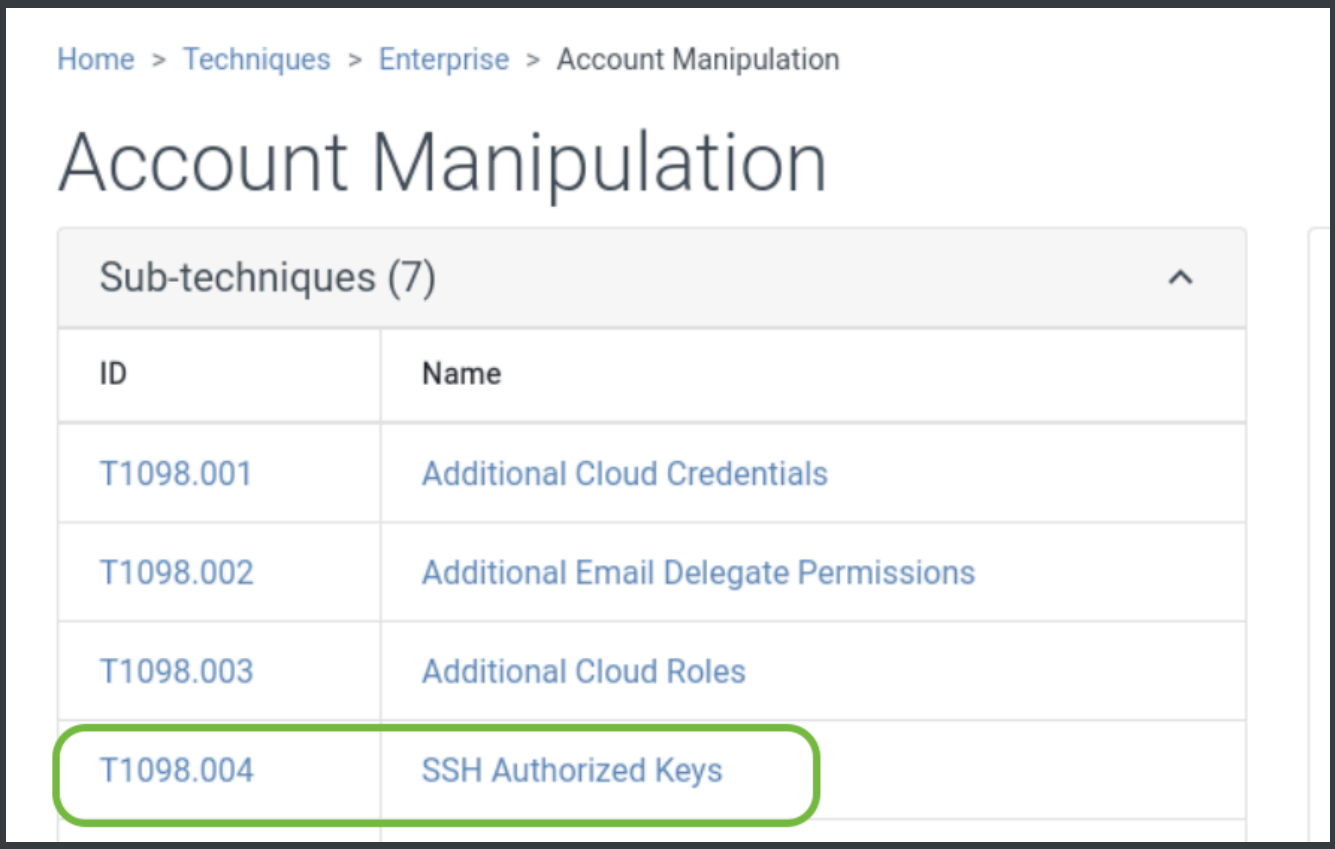


As we saw in question 8, this key was being copied into the file `/home/ubuntu/.ssh/authorized_keys`.

Answer: `/home/ubuntu/.ssh/authorized_keys`

10. **Can you confirm the MITRE technique ID of this type of persistence mechanism?**

    Consulting the MITRE ATT&CK Enterprise Matrix for the Persistence Tactic we can see the Technique 'Account Manipulation' which has the Sub-Technique of 'SSH Authorised Keys'.



The sub-technique ID is T1098.004

Answer: `T1098.004`